Water Volumes Supplied to Regions in Istanbul Forecast

Fikret Bartu Hıraekşi

Tools used:

Python 3

Jupyter Notebook

Libraries used:

Pandas (Data manipulation)

Numpy (Numerical functions)

Sklearn (Machine Learning algorithms)

Seaborn (Visualization)

Plotly (Visualization)

Statsmodels (Statistical tools)

Introduction

A time series is a sequence of data points recorded at successive, equally spaced points in time. Time series analysis is a type of data analysis that analysts use to understand past trends, identify patterns, detect anomalies, and perform forecasting to predict the future. It can be used to track the stock market to spot trends and define a strategy, in cardiology to analyze the recorded electrical activity of a patient’s heart over seconds or minutes to detect anomalies, and in inventory management to identify seasonal spikes and prevent stockouts.

This is a special passion project for me. I haven't been introduced to time series for a very long time, but I decided to learn it spontaneously after becoming interested in the topic. This project forecasts the water supplied to various regions in Istanbul by İSKİ, using data from the Istanbul Metropolitan Municipality's open dataset portal.

The project forecasts the supplied water for the months between 2024 and 2026, which is missing from the dataset. Unfortunately, I couldn't find the actual water supply figures for the years between 2024 and 2026 because the dataset hasn't been updated. However, with the help of this project, I now have a strong idea of what those numbers might be!

Thank you in advance for reading about my project.

Understanding the Data

	_id	Ay	2009.0	2010.0	2011.0	2012.0	2013.0	2014.0	2015.0	2016.0	2017.0	2018.0	2019.0	2020.0	2021.0	2022.0	2023.0	2024.0
0	1	Ocak	55926	60626	65178	67137	71538	74528	77053	80206	81757	84212	83576	84977984	84160554	88812431	90242130	91313882.0
1	2	Şubat	50838	55607	59159	63613	64379	66844	67912	74114	73515	75692	74797	80120814	77068001	80043139	80459071	NaN
2	3	Mart	57261	60690	65895	67727	71663	73786	75459	79147	81215	84274	84666	88457622	85420000	87987877	89652551	NaN
3	4	Nisan	56034	60060	63179	66769	70859	72909	74177	80340	80663	84239	84968	84337847	82501000	87155031	85804024	NaN
4	5	Mayıs	62878	68131	69283	71704	80496	79584	82905	84951	87441	90369	93692	90040181	91395884	95320559	91639400	NaN
5	6	Haziran	65211	65795	72126	77507	79433	80020	82472	88033	87008	90831	93135	90923882	89492561	96060213	94373679	NaN
6	7	Temmuz	66197	68314	76919	83574	84224	84905	88453	88076	93177	95090	96028	98185057	96387379	96961473	103709735	NaN
7	8	Ağustos	66583	75594	75957	80098	82987	83262	89859	91773	91794	93402	94171	98344547	103430618	100430578	103457024	NaN
8	9	Eylül	61154	66734	71906	76396	80163	78215	84083	84767	87630	88053	91987	95694061	95039208	95183779	98550379	NaN
9	10	Ekim	61555	66419	70069	75991	76077	77767	82263	85454	87114	86864	90439	91975770	91950106	95084476	95677961	NaN
10	11	Kasım	59452	64087	65797	70456	73246	75344	79387	80802	84163	83213	86747	85450631	87657249	89666854	89850472	NaN
11	12	Aralık	60567	65969	68282	71964	74388	77285	81133	80958	85165	84728	51278	85625581	89488145	90965659	93647690	NaN

Diagnosis

As one can see, this dataset has several problems.

- The dataset is in wide format, where months are distributed across rows and years are spread across columns.
- The value for December 2019 is unusually low (51,278), unlike the other December values. Furthermore, looking at news from that period, there are no reports indicating a water supply crisis in Istanbul.
- Different units were recorded between 2009–2019 and 2020–2024. From 2009 to 2019, the water supply was recorded in tens of thousands, whereas after 2019, it was recorded in tens of millions. It seems that a different decision was made regarding data entry.
- Month values are missing except the January of 2024.
- The year values have '.0' in their names, and they are not a datetime data type.

So, in order to benefit from this dataset, we have to turn it into proper time series data. This dataset is also univariate, meaning it has only one predictor: the tons of water supplied. In contrast, multivariate datasets consist of more than one predictor, such as temperature data combined with water supply. Additionally, there are 181 non-null data points, which is quite good for a project of this scope.

Cleaning

- *The dataset was melted from wide format into long format, while the 'ID' and 'Month' columns remained constant and the year columns were combined into a single column named 'Year'.*

- *The '.0's removed from the years.*

- *The text values of the months were replaced with their numerical equivalents (January = 1, February = 2, etc.)*

- *Combining with the year and month number the final date column has been created.*

- *The frequency was set to the start of the month (the first day of each month).*

- *The values between 2009 and 2019 were multiplied by 1000 to convert the units into tens of millions.*

- *Months with null values removed.*

```
Ratios for the December divided by Neighbor Average is:[1.01, 1.02, 1.03, 1.01, 1.01, 1.01, 1.02, 1.0, 1.01, 1.02, 1.01, 1.01, 1.01, 1.03]
Median value of the ratio is: 1.0139122451264146
Imputed value of the December 2019 data is: 87057032.03586882
```

- *Ratios for the December values were determined (each December value divided by the average of its neighbors, November and January). As one can see, the ratios fluctuate between 1.01 and 1.03, remaining very consistent. Thus, the median ratio is 1.01. Consequently, the value for December 2019 was replaced with the median ratio multiplied by the neighbor average for December 2019.*

Trends & Seasonality and Residuals

Overview

Now, we have a proper time series dataset to inspect further. Here are the components to understand the better.

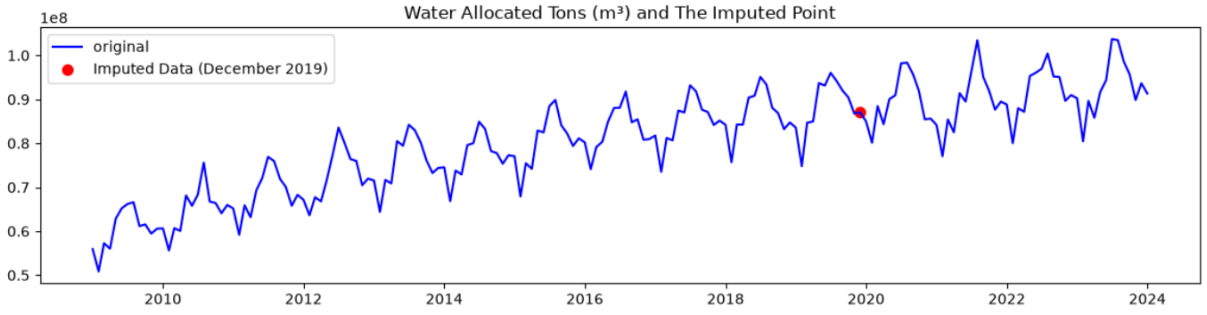
First recorded date: 2009-01-01 00:00:00
Last recorded date: 2024-01-01 00:00:00

Average Tons (m³) of Water Allocated By Year

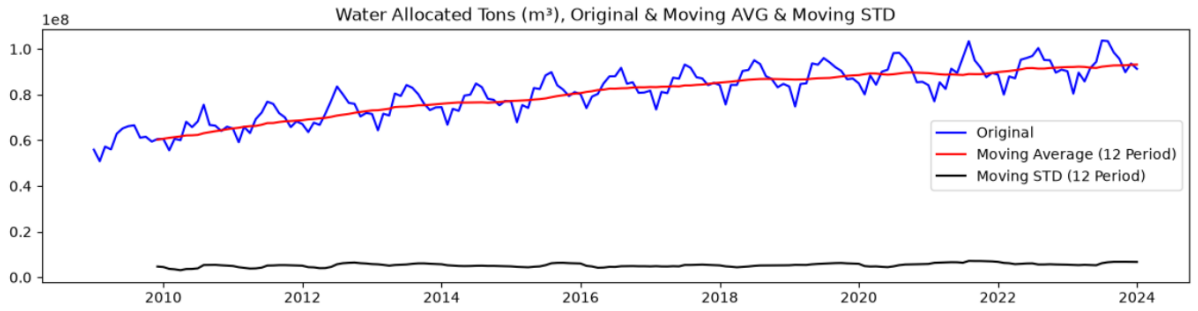
2009	60304666.67
2010	64835500.00
2011	68645833.33
2012	72744666.67
2013	75787750.00
2014	77037416.67
2015	80429666.67
2016	83218416.67
2017	85053500.00
2018	86747250.00
2019	88438586.00
2020	89511164.75
2021	89499225.42
2022	91972672.42
2023	93088676.33

Average Tons (m³) of Water Allocated By Month

1	77577748.81
2	69610735.00
3	76886736.67
4	75599660.13
5	82655334.93
6	83494755.67
7	88013376.27
8	88742851.13
9	83703695.13
10	82313354.20
11	78354613.73
12	77429605.00



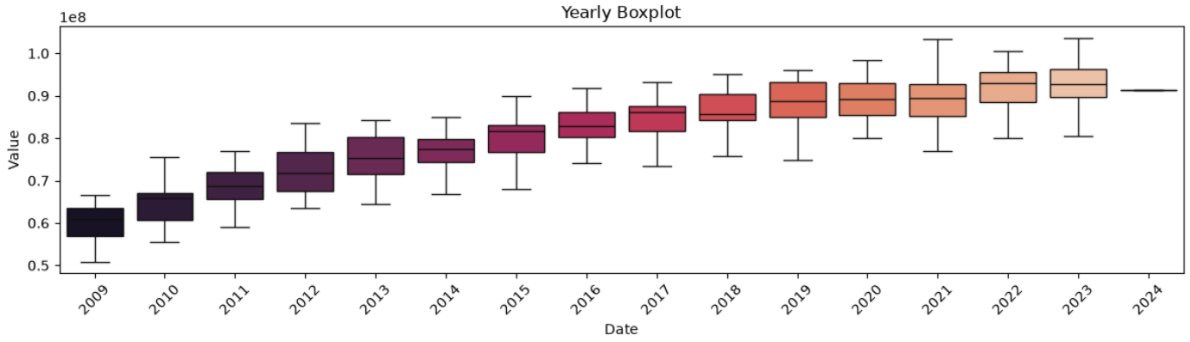
The imputed value (December 2019) fits the time series well. No strange, sudden drops or rises were detected.



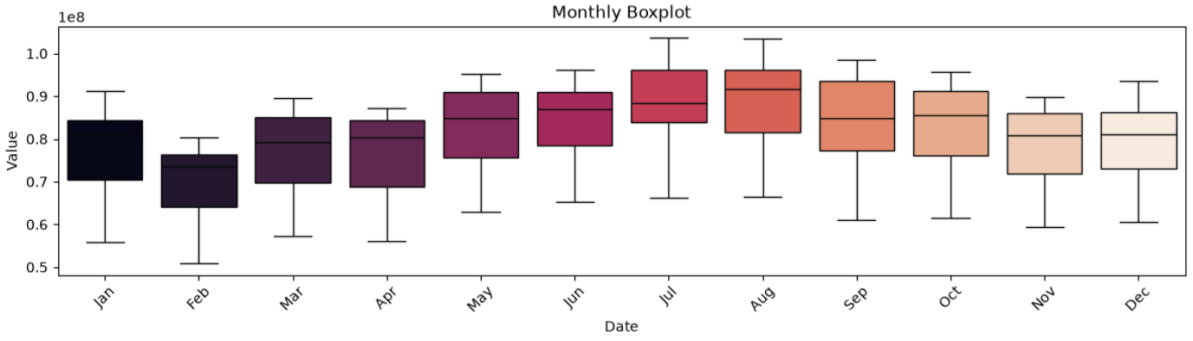
The original data, shown in blue, shows strong seasonality. This makes sense because water demand increases during the hot summer months and slowly decreases as temperatures drop.

The moving average data, shown in red, shows the trend cleansed of seasonality. As we can see, there is an upward trend in the water supplied because Istanbul's population has significantly increased. However, the trend does not always grow at the same rate; it starts to taper off after 2020 and flattens out.

The moving standard deviation data, shown in black, shows the data variation. The standard deviation line stays mostly flat, which indicates low variation between points.



The upward trend and fall off after 2020 can be seen more clearly with this box plot.

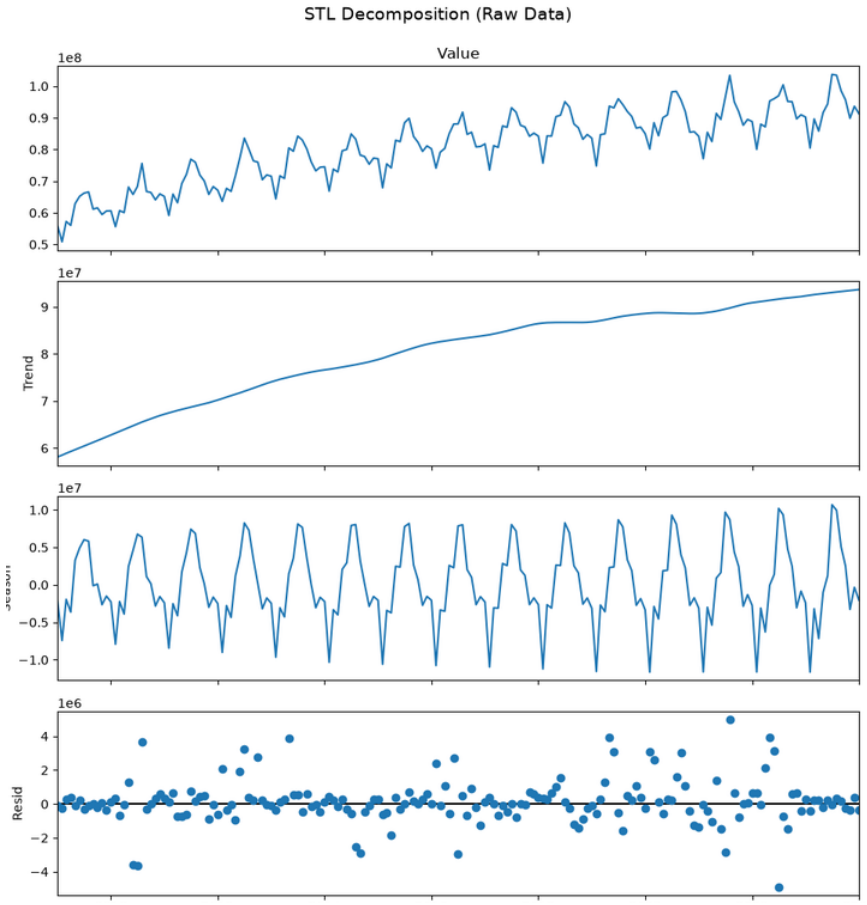


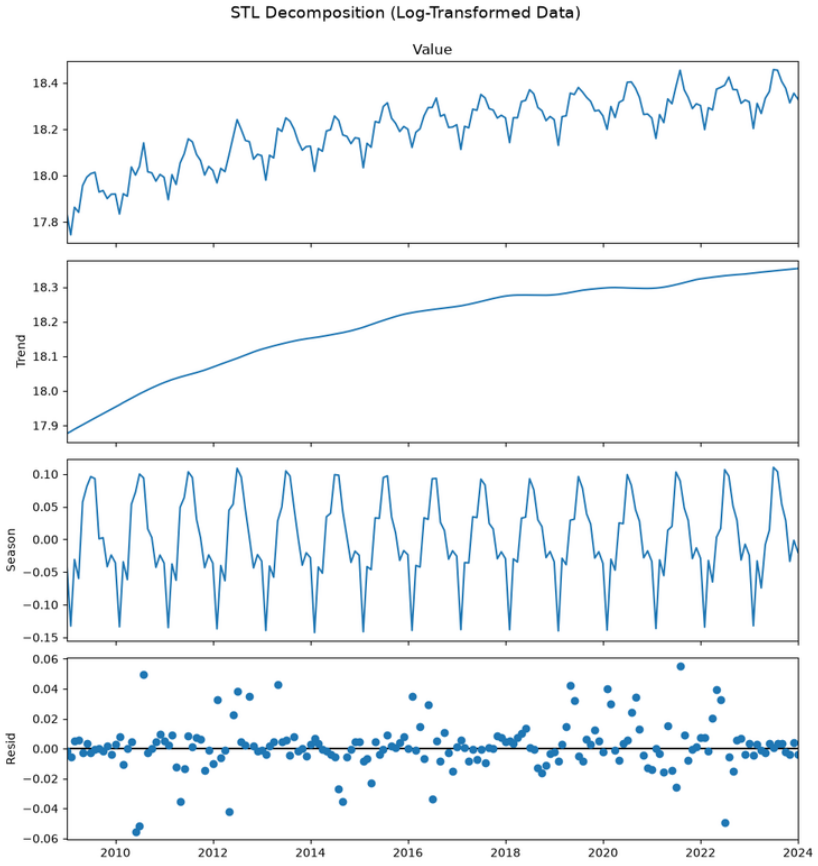
July and August have the highest water supply on average. This confirms that the summer months see the highest water usage. The variance on the months July and August is also bigger than other months.

Decomposition

The graphics are not enough for us to understand the depths of the data. Especially, understanding if the seasons are additive or multiplicative is important for methods like Holt-Winters.
In additive seasonality the seasonal fluctuations remain constant in size over time, regardless of the overall trend level.
Otherhand, in multiplicative seasonality the seasonal fluctuations scale proportionally with the overall trend. The peaks and troughs widen over time.

The STL (seasonal-Trend decomposition using LOESS) analysis assumes that our series is additive. Which assumes that seasonal fluctuations remain constant in absolute size.
To understand the structure of seasonality, we will first decompose the original raw data and the logarithmically transformed data. The logarithm stabilizes the variance and makes our multiplicative data compatible with STL, which assumes additive data. If we can see a multiplicative seasonality in raw, and constant additive seasonality in log data, it means that there is a multiplicative seasonality. If there is an additive seasonality in raw and log, then it is additive.





While the seasonal peaks increase with each passing season in the raw data, they remain very consistent in the log data. To understand the sizes better, I calculate the lowest and the highest (peak) point of every season in both STL.

Raw STL - (13444849.745, 14678006.851, 15867588.08, 17262544.761, 17769250.53, 18369251.564, 18777528.293, 18765700.447, 18996768.053, 19481639.2, 20233520.696, 20931290.282, 21303315.616, 21816322.987, 22328009.473)

Log STL - (0.229, 0.234, 0.239, 0.246, 0.244, 0.242, 0.239, 0.233, 0.230, 0.231, 0.236, 0.238, 0.240, 0.241, 0.243)

As expected, the seasonal peaks increased 13444849 to 22328009 in the raw data while it fluctuates around 0.229 to 0.243 in log data. So, it is indeed multiplicative.

Stationarity Test

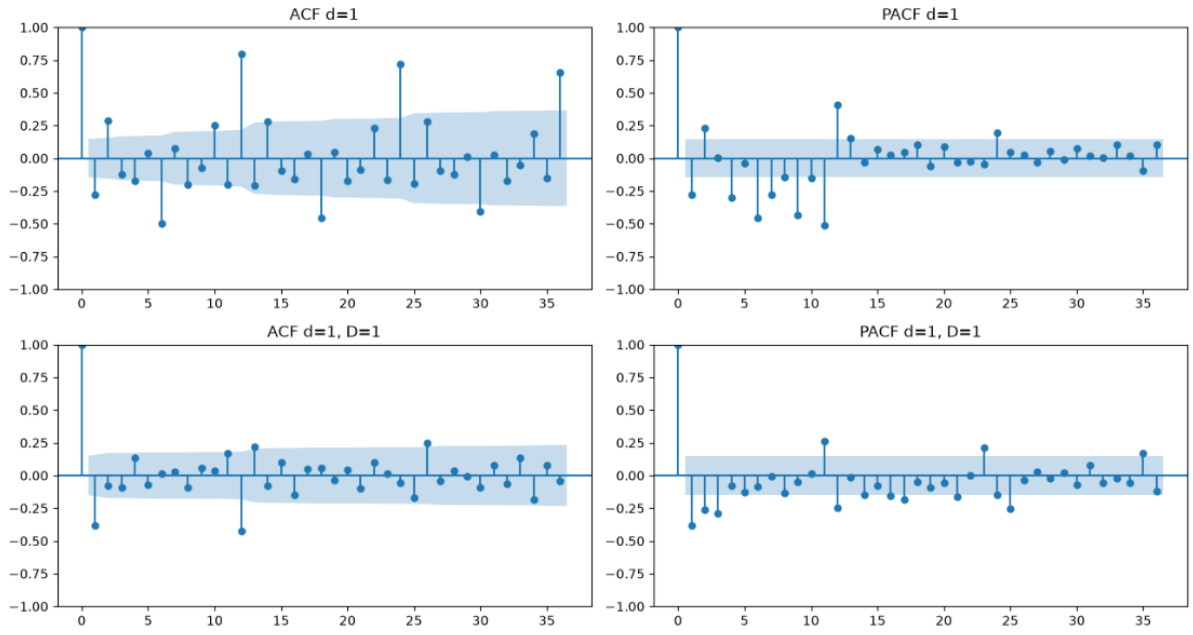
Stationarity is one of the key elements in time series analysis. A time series is considered stationary if its mean, variance and autocorrelation remain constant over time. It is important because most forecasting models like ARIMA need stationary data to make accurate predictions. We will use two tests: Augmented Dickey–Fuller test and Kwiatkowski-Phillips-Schmidt-Shin test on 4 different models.

log taken **-** Logarithmic transformed data to stabilize variance.
d=1 **-** Subtracting the previous period's value from the current value. This removes the long-term trend
D=1 **-** Subtracting the value from the exact same season of the previous year. This removes repeating seasonal patterns, such as the regular summer surges in water consumption.
d = 1, D = 1 **-** Applying both regular differencing and seasonal differencing together.

Results**:**

- ***log taken: Non-stationary*** (ADF $p > 0.05$ and KPSS $p < 0.05$). Neither trend nor seasonality is removed.
- ***d=1: Stationary*** according to both tests (ADF $p = 0.016$, KPSS $p = 0.100$). Removes the overall trend.
- ***D=1: Non-stationary*** (KPSS fails at $p = 0.010$). Removing only seasonality is not enough because the trend remains.
- ***d=1, D=1: Stationary*** (ADF $p = 0.000$, KPSS $p = 0.100$). This combination successfully eliminates both the trend and seasonality.

ACF/PACF



ACF measures the total correlation between a time series and its past values (lags). It shows a chain reaction, meaning that looking at this month's relationship with the value from three months ago also incorporates the relationships with one and two months ago.

PACF measures the direct correlation between a time series and a specific lag value, after removing the effects of all intervening lags in between.

We removed the trend using first-degree differencing. However, both ACF and PACF show spikes at lags 12, 24, and 36, indicating a yearly seasonality. By combining seasonal differencing with first-degree differencing, the spikes are successfully knocked down and brought to the blue confidence band, making it much more effective than taking only the first-degree difference. This step is especially important for determining the d and D parameters in the ARIMA model. And it shows d=1, D=1.

MODELS

Baseline

Our first model is the Seasonal Naive. It is a very simple model and it serves as a baseline that every model needs to surpass. Seasonal Naive generates a forecast by repeating the pattern of the last 12 months from the training data.

We split the data as 2009 January - 2021 December and 2022 January to 2024 January.

Then, the performance of the Seasonal Naive has been calculated as:

Mean Absolute Error (MAE): 3,436,327.32
Root Mean Squared Error (RMSE): 3,984,961.94
Mean Absolute Percentage Error (MAPE): 3.73%

Holt-Winters

Holt Winters is a very classical time series forecasting method that is designed specifically for datasets that exhibit both a trend and seasonality just like this dataset.
Holt Winters method accounts for three levels of the data.
Level, Trend and Seasonality.
It smoothes level by controlling how quickly the baseline adapts to recent changes.
It smoothes the trend by controlling how fast the slope/trend updates based on recent trajectory.
It smoothes the seasonality by controlling how quickly the seasonal component adapts to new seasonal patterns.
With giving a 'multiplicative' parameter to seasonality in Holt-Winters method and a damped parameter for flatting the trend in the long term, i measure the method's performance as:

Mean Absolute Error (MAE): 1,277,774.57
Root Mean Squared Error (RMSE): 1,644,419.51
Mean Absolute Percentage Error (MAPE): 1.36%

Also, the trained model gives the parameters it used as:
smoothing_trend = 0.0, ***smoothing_seasonal = 0.0*** (The model relies on initial baseline patterns rather than updating them with recent noise, indicating highly stable and predictable seasonal behavior.)

smoothing_level ≈ 0.22 (The baseline updates smoothly without overreacting to short-term fluctuations.)

damping_trend ≈ 0.993 (The damped trend feature didn't do much work. It flattened very slightly. Could cause problems in the long-run predictions. Could boost up the forecast with the trend's speed.)

ARIMA

ARIMA stands for Autoregressive Integrated Moving Average. It is the most popular model between classical models of time series. Unlike simple baseline models, ARIMA looks deep into the statistical structure of a sequence to model its underlying patterns. For further information on AR, I and MA components: https://www.geeksforgeeks.org/machine-learning/model-selection-for-arima/

I found the difference parameters d and D as 1 in the early part. So in order to find an optimized model, I searched $[0, 1, 2]$ for p and q, and $[0, 1]$ for P and Q, trying a total of 36 different models with every possible combination of these parameters.

order	seasonal	AIC	BIC	MAE	RMSE
(1, 1, 1)	(0, 1, 1, 12)	-672.520625	-660.669246	1283612.78	1868445.94
(0, 1, 2)	(0, 1, 1, 12)	-672.511425	-660.660046	1297624.60	1891641.50
(0, 1, 1)	(0, 1, 1, 12)	-672.233065	-663.344531	1357448.99	1968995.54

These three models have the almost same AIC score. So, between them, I choose the model with the lowest BIC.

AIC focuses on predictive accuracy. It tries to find the model that will give better forecasts. On the other hand, BIC focuses on finding the true underlying model. BIC has a higher penalty for model complexity. So, having almost the same AIC as the first two, I also picked the simpler model of the two, because simpler is often better. So the performance of this model is:

Mean Absolute Error (MAE): 1,357,448.99

Root Mean Squared Error (RMSE): 1,968,995.54

Mean Absolute Percentage Error (MAPE): 1.47%

```
                                SARIMAX Results
==========================================================================================
Dep. Variable:                             Value   No. Observations:                  156
Model:             SARIMAX(0, 1, 1)x(0, 1, 1, 12)   Log Likelihood                 339.117
Date:                            Tue, 20 Oct 2026   AIC                           -672.233
Time:                                    17:48:13   BIC                           -663.345
Sample:                                01-01-2009   HQIC                          -668.621
                                     - 12-01-2021
Covariance Type:                             opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ma.L1         -0.6521      0.079     -8.249      0.000      -0.807      -0.497
ma.S.L12      -0.7899      0.101     -7.844      0.000      -0.987      -0.593
sigma2         0.0005   5.62e-05      8.336      0.000       0.000       0.001
===================================================================================
Ljung-Box (L1) (Q):                   0.42   Jarque-Bera (JB):                15.81
Prob(Q):                              0.52   Prob(JB):                         0.00
Heteroskedasticity (H):               0.83   Skew:                             0.54
Prob(H) (two-sided):                  0.52   Kurtosis:                         4.22
===================================================================================

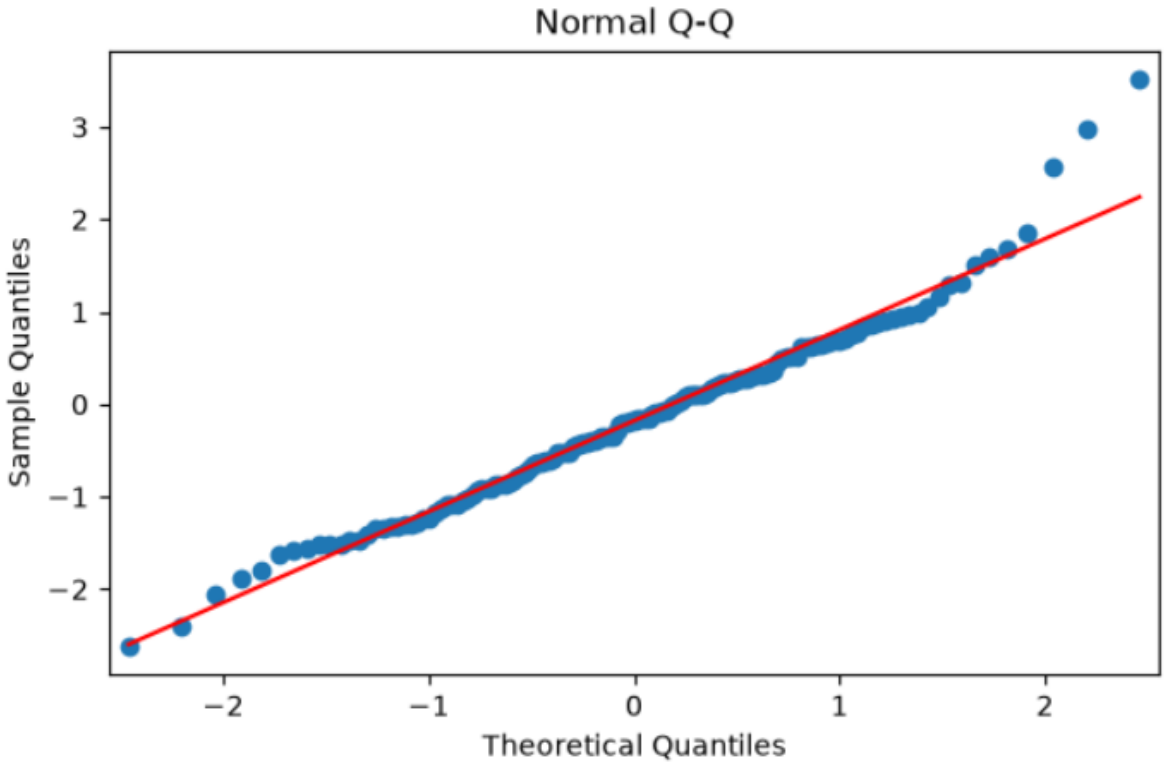
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
      lb_stat  lb_pvalue
12  13.273436   0.349483
24  25.107640   0.399930
```

This is the SARIMA diagnosis of the selected model.

MA terms are significant so the model is valid. The seasonal MA is strong as expected.

Ljung-Box scores being 0.349483 at lag 12 and 0.399930 in lag 24, both well above 0.05 results that there is no leftover autocorrelation.

Heteroskedasticity being 0.83 p value of 0.52 means constant variance, which is the result of taking a log.




```
Date
2010-08-01    0.082272
2021-08-01    0.076260
2010-06-01    0.066223
2012-07-01    0.058799
2016-07-01    0.057196
```

The only problem is that there are months that supply came in well above what the model expected.

When looking at such data, we can see that the model under-estimates the summer months. This is a side note to remember and a weakness of our model. ARIMA sometimes couldn't be able to estimate how much the value increases.

Rolling-Origin Test

In order to validate the performance of our model, now it is time to use the rolling-origin test. It is very similar to the cross validation in classical machine learning, but for time series analysis. It starts with a training window and test on the immediate next period. After calculating the performance metrics of that period, test rolls the origin forward by a step and test on the next period. It repeats this process across multiple steps.

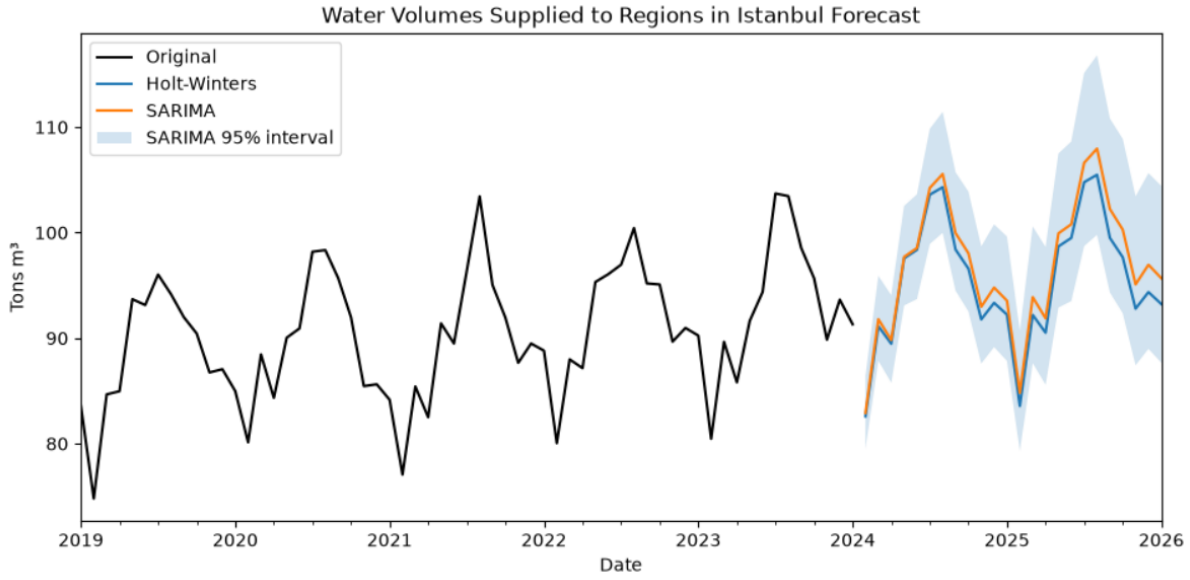
I define cutoff points from 2017 to 2022. And measure the average of the performance metrics as:

Evaluation Cut-off (points)	Seasonal Naive (MAE)	Holt-Winters (MAE)	SARIMA (MAE)
2017-12-01	1,966,583.33	1,362,160.70	2,313,142.25
2018-12-01	1,946,502.67	1,422,270.74	1,172,450.06
2019-12-01	2,609,397.09	1,959,537.80	2,371,245.48
2020-12-01	2,097,098.67	2,254,548.72	2,255,943.76
2021-12-01	2,973,453.67	906,318.16	934,680.40
2022-12-01	2,235,787.25	1,596,484.96	1,634,691.19
Mean Absolute Error (Overall)	**2,304,803.78**	**1,583,553.51**	**1,780,358.86**

Holt-Winters is best in 4 of 6 (2017, 2019, 2021, 2022). In 2020, it's tied with SARIMA and beaten by the naive forecast. SARIMA wins once clearly in 2018. In 2021 and 2022, it is within about 40k of Holt-Winters, which is a tie in practice. In 2017, SARIMA is much worse, and that single window drives most of the gap in the average.

Overall, they both surpass the naive baseline except in 2020 (possible COVID effect), but there is no significant difference between them. So, it is better to make a forecast using both of them.

Final Forecast



This is the final model with forecasts from the two models. A table consisting of all the forecast values and the 95% CI values for SARIMA can be found in the notebook, directly above this graph. The table was not included in the PDF because it is too long. This forecast is for the years between 2024 and 2026, trained on all data prior to 2024.